

34 体 AI エージェント 導入手順と使い方ガイド

Claude Code に、役割の違う 34 体の専門エージェントと 13 個のモードを組み込むための手順と使い方です。前半が導入の手順、後半が使い方です。

導入の手順

導入に使うのは、別にお渡しする **AGENT34-KIT.md** というファイルです。この Word 文書は読むためのもので、導入には使いません。

手順	やること
1	AGENT34-KIT.md をパソコンに保存する
2	Claude Code を開き、そのファイルを添付する
3	「このキットの手順どおりに構築してください」と入力して送信する
4	完了の報告が出たら、Claude Code を一度閉じて開き直す
5	「34 体チェック」と入力する（動作の点検が自動で走る）

導入で起きること

- ・ 専門エージェント 34 体、スキル 15 個、運用ルール、使い方ガイドが、Claude の設定フォルダに入ります
- ・ すでにある同じ名前のファイルは、置き換える前に自動で退避されます。ご自身で書いた設定は消えません
- ・ 導入のあと、ファイルの検証と個人情報の点検が自動で行われます
- ・ 専用の機器、追加のソフト、ネットワークへの接続は要りません
- ・ Windows では、設定フォルダ **~/.claude** は **%USERPROFILE%\claude** のことです

1. まず試す 3 つ

チャット欄にそのまま入力します。

入力	起きること
deep interview 社内向けの問い合わせ管理ツールを作りたい	1 問ずつ質問され、最後に要件のまとめが出る
ralplan このフォルダのコードにログイン機能を足したい	計画が示され、承認するまで実行されない
explore でこのフォルダの構成を調べて	調査専門のエージェントが調べて報告する

2. 単体で使う（エージェントを名指しする）

「〇〇で～して」「〇〇に～を頼んで」と名前を入れて依頼します。名前だけで呼べます。

`debugger` でこのエラーの原因を調べて
`security-reviewer` にこの変更を見てもらって
`writer` で README を書いて

「編集」が「不可」のエージェントは、調べて報告するだけでファイルを書き換えません。安心して調査やレビューを任せられます。

エージェント	分類	得意なこと	区分	編集
<code>analyst</code>	計画・設計	要件の抜け・隠れた前提・範囲外を洗い出す	opus	不可
<code>planner</code>	計画・設計	手順と完了条件を組み立て、実行計画を作る	opus	可
<code>architect</code>	計画・設計	設計判断、構造の見直し、難しい不具合への助言	opus	不可
<code>critic</code>	計画・設計	計画や案の弱点を多角的に批判・点検する	opus	不可
<code>executor</code>	実装	指示された変更を最小の差分で実装する	sonnet	可
<code>debugger</code>	実装	不具合・ビルドエラーの根本原因を突き止めて直す	sonnet	可
<code>tracer</code>	実装	複数の仮説を証拠で比べ、原因をたどる	sonnet	可
<code>refactorer</code>	実装	大規模な構造の組み替えと技術的負債の解消	opus	可
<code>code-simplifier</code>	実装	動作を変えずにコードを読みやすく整える	opus	可
<code>migrator</code>	実装	フレームワーク更新や仕様変更に伴う移行	sonnet	可
<code>optimizer</code>	実装	計測にもとづく速度・メモリの改善	sonnet	可
<code>verifier</code>	品質	完了条件を証拠で確かめ、合否を判定する	sonnet	可
<code>qa-tester</code>	品質	実際に動かして操作し、動作を確かめる	sonnet	可
<code>test-engineer</code>	品質	テスト方針の設計とテストの作成・安定化	sonnet	可
<code>code-reviewer</code>	品質	重要度つきのコードレビュー（正しさ・保守性）	opus	不可
<code>security-reviewer</code>	品質	脆弱性・秘密情報・危険な書き方の点検	opus	不可
<code>risk-assessor</code>	品質	変更の影響範囲と戻しにくさの評価	opus	不可
<code>compliance</code>	品質	個人情報保護・決済・監査などの規制対応の確認	opus	不可
<code>accessibility</code>	品質	画面のアクセシビリティ（WCAG）の点検と修正案	sonnet	一部

エージェント	分類	得意なこと	区分	編集
database	専門領域	テーブル設計、クエリ改善、移行計画	sonnet	可
api-designer	専門領域	API の設計、版管理、仕様書づくり	sonnet	可
devops	専門領域	自動テスト・配備の仕組み、コンテナ構成	sonnet	可
monitor	専門領域	ログ・計測・追跡・通知の設計	sonnet	可
data-pipeline	専門領域	データの取り込み・加工・連携処理	sonnet	可
mobile	専門領域	iOS・Android・React Native の開発	sonnet	可
designer	専門領域	画面と操作のデザインと実装	sonnet	可
ux-researcher	専門領域	使いやすさの分析、つまづく箇所の特定	sonnet	不可
prompter	専門領域	AI への指示文（プロンプト）の設計と改善	sonnet	可
scientist	専門領域	データ分析と統計にもとづく検証	sonnet	不可
explore	調査・文章	ファイルやコードを素早く探して全体像をつかむ	haiku	不可
document-specialist	調査・文章	公式文書・外部資料を調べて根拠を示す	sonnet	不可
writer	調査・文章	README・手順書・説明文を書く	haiku	可
localization	調査・文章	多言語対応と翻訳、地域ごとの調整	haiku	可
git-master	調査・文章	コミットの整理、履歴の管理	sonnet	可

「編集」の「一部」は、既存のファイルの修正はできるが、新しいファイルの作成はできないという意味です。

選び方の早見

やりたいこと	頼む相手
作る・直す	executor
原因を突き止める	debugger（絡み合った原因は tracer）
探す	explore
決める・設計する	architect
段取りを組む	planner
要件の抜けを見つける	analyst
計画や案を批判してもらう	critic
出来上がりを確かめる	verifier（実際に動かすなら qa-tester）
コードを見してもらう	code-reviewer（安全面は security-reviewer）
文章を書く	writer

3. モードで使う（キーワード）

キーワードを文の中に入れると、決まった手順で進みます。先頭に / を付けて `/ralph` のように呼ぶこともできます。

キーワード	ひとことで	向いている場面
<code>ultrawork (ulw)</code>	並列で一気に進める	独立した作業がいくつもあるとき
<code>ralph</code>	合格するまで繰り返す	途中で止まらず、やりきってほしいとき
<code>autopilot</code>	要件から成果物まで通して進める	まるごと任せたいとき
<code>ralplan</code>	計画を練り、承認を得てから動く	実行前に計画を確認したいとき
<code>team</code>	役割を決めて分担する	複数の作業を担当制で進めたいとき
<code>tdd</code>	テストを先に書く	既存の動きを壊したくないとき
<code>deslop</code>	AI っぽさ・無駄を取り除く	文章やコードをすっきりさせたいとき
<code>deep interview</code>	質問で要件を固める	何から始めるか決まっていないとき
<code>deep-analyze</code>	変更せずに原因と影響を調べる	直す前に状況をつかみたいとき
<code>deepsearch</code>	漏れなく探す	関係する箇所を全部洗い出したいとき
<code>ultrathink</code>	重要な判断を考え抜く	技術選定や比較検討をするとき
<code>ccg</code>	3つの視点を突き合わせる	設定方法や方針を多面的に確かめたいとき
<code>cancelomc</code>	すぐ止める	止まらない・思っていない動きをしているとき

キーワードの意味を尋ねるだけ（「ralph とは？」など）のときは、起動せずに説明が返ります。

使い分けのコツ

- 何を作るか決まっていない → `deep interview`
- 決まっているが、いきなり実行されるのは不安 → `ralplan`
- 計画はできた、あとは速く → `ultrawork`
- 途中で止まらず最後まで → `ralph`
- まるごと任せたい → `autopilot`
- とにかく止めたい → `cancelomc`（返事が返らないときは Esc キー）

4. 組み合わせて使う（コンビネーション）

キーワードを記号でつなぎます。

- **A → B** … A が終わったら B（矢印は `->` と打っても通じます）
- **A + B** … A と B を同時に効かせる
- **+エージェント名** … そのエージェントを必ず使う

`deep-analyze → ralplan → tdd` 決済処理に返金機能を足したい
`ultrawork + ralph` この一覧にある修正を全部終わらせて
`deep-analyze → ralplan → ultrawork +optimizer` 一覧画面の表示が遅い

定番の組み合わせ

場面	組み合わせ
不具合を直す	<code>deep-analyze → ralplan → autopilot + ralph</code>
新機能を足す（既存を壊さない）	<code>deep-analyze → ralplan → tdd</code>
計画を確認してから安全に足す	<code>ralplan → tdd</code>
大規模・多ファイルの実装	<code>ultrawork + ralph + autopilot</code>
リファクタリング	<code>deepsearch → deep-analyze → ultrawork</code>
知らないコードを把握する	<code>deepsearch → deep-analyze → team</code>
遅い処理を速くする	<code>deep-analyze → ralplan → ultrawork +optimizer</code>
安全性を点検する	<code>deep-analyze → ralplan → autopilot +security-reviewer</code>
API を設計して作る	<code>ralplan → tdd → ultrawork +api-designer</code>
データベースを設計・移行する	<code>deep-analyze → ralplan → autopilot +database</code>
自動テスト・配備の仕組みを作る	<code>ralplan → autopilot + ralph +devops +risk-assessor</code>
画面をデザインする	<code>deep interview → ralplan → ultrawork +designer</code>
提案書・企画をゼロから作る	<code>deep interview → ultrathink → deslop</code>
急ぎで資料を作る	<code>deep interview → ultrawork → deslop</code>
文章の質を上げる	<code>deslop → ultrathink</code>
技術文書・README を作る	<code>ralplan → ultrawork +writer</code>
比較して決める	<code>deep-analyze → ultrathink</code>
要件があいまいな新規案件	<code>deep interview → ralplan → team + ultrawork</code>
データを分析して報告する	<code>deep-analyze → ultrawork +scientist</code>
止まらない・暴走している	<code>cancelomc</code> 、止めたあとに <code>deep-analyze</code>

組み合わせを自分で作るとき の型

「調べる → 決める → 作る → 確かめる」の順に、必要な段だけ並べます。

段	使うもの
調べる	deepsearch、deep-analyze、deep interview
決める	ralplan、ultrathink、ccg
作る	tdd、ultrawork、autopilot、team
確かめる・仕上げる	ralph、deslop、+verifier、+code-reviewer

小さな作業に長い組み合わせを使うと、時間と費用だけが増えます。1 ファイルの軽い修正や単純な質問は、キーワードなしで頼むのが一番速いです。

5. 繰り返して仕上げる（ループの設計）

1 回で完璧に仕上げようとせず、「作る → 確かめる → 直す」を回して合格まで持っていく考え方です。うまく回すには、始める前に次の 4 つを決めます。

決めること	例
完了条件（確かめられる形で）	テストが全部通る／画面に合計金額が出る
確かめ方	テストを実行する／実際に画面を開く
回数の上限	10 周まで
止め方	cancelomc と入力する

3 種類のループ

(1) 合格まで回す — ralph

ralph テストが全部通るまで直して。上限は 8 周

完了条件を決め、不合格の項目だけを直しながら繰り返します。同じ失敗が 3 周続くと、自分で止まって報告します。

(2) 計画を練り直す — ralplan

計画係・設計係・批判係が計画を回し読みして磨きます。実行前の段階で回すので、手戻りが減ります。

(3) 一定の間隔で繰り返す — /loop

Claude Code に備わっている機能で、同じ依頼を決まった間隔で実行します。

```
/loop 5m テストを実行して、失敗があれば原因を報告して
/loop 変更のたびに画面の崩れを点検して
```

間隔（5m は 5 分）を省くと、Claude が間隔を自分で決めます。止めるときは `cancelomc` と入力するか、Esc キーを押します。お使いのバージョンに `/loop` が無いときは、`ralph` で代用できます。

ループを組み合わせる

ralplan → tdd → ralph 在庫の引き当て処理を作りたい

計画を練り（ループ 2）、テストを先に書き、合格まで回す（ループ 1）流れです。品質を重視する作業の基本形として使えます。

うまく回らないとき

症状	見直すところ
いつまでも終わらない	完了条件があいまい。確かめられる形に書き直す
同じ所で失敗し続ける	いったん <code>cancelomc</code> 。 <code>deep-analyze</code> で原因を調べてからやり直す
直すたびに別の所が壊れる	<code>tdd</code> を足して、壊れたらすぐ分かるようにする
費用がかさむ	回数の上限を下げる。範囲を小さく区切る

6. 費用と速さの目安

エージェントは 3 つのモデル区分に分かれています。

区分	特徴	主な用途
haiku	速くて安い	検索、短い文章
sonnet	標準	実装、調査、検証
opus	高精度で高い	設計、深い分析、重要なレビュー

- 並列（`ultrawork`、`team`）は速く終わりますが、同時に動く数だけ費用がかかります
- 繰り返し（`ralph`、`/loop`）は、周回の数だけ費用がかかります。上限を決めて使います
- 契約プランによっては `opus` が使えない、または上限に届きやすいことがあります。その場合は `~/.claude/agents/` の該当ファイルを開き、`model: opus` を `model: sonnet` に書き換えてください

7. 困ったとき

症状	対処
エージェント名を言っても使われない	Claude Code を終了して開き直す。そのあと「34 体チェック」と入力する

症状	対処
キーワードが効かない	<code>/ralph</code> のように <code>/</code> を付けて呼ぶ。出てこなければ開き直す
opus のエージェントだけ失敗する	6 の方法で <code>sonnet</code> に書き換える
止まらない	<code>cancelomc</code> 。返事が無ければ Esc キー
入れた直後の状態に戻したい	「34 体チェック」と入力すると、欠けや書き換えを見つけて直す
自分のルールを足したい	<code>~/.claude/CLAUDE.md</code> の <code>AGENT34:START</code> ~ <code>AGENT34:END</code> の外側に書く（内側は入れ直すと上書きされる）

8. 元に戻す・取り除く

- 導入時に上書きされたファイルは `~/.claude/agent34-kit/backup-日時/` に残っています。戻したいものをそこからコピーします
- 取り除くときは Claude Code に次のように頼みます: 「AGENT34 キットを取り除いて。
`~/.claude/agent34-kit/payload/manifest.tsv` に載っているエージェントとスキルを消し、`~/.claude/CLAUDE.md` の AGENT34 ブロックを消して。消す前に一覧を見せて」

出どころ

34 体のうち中心となる定義は、公開されているオープンソース「oh-my-claudecode」（MIT ライセンス）に由来します。ライセンスの表記は同じフォルダの `NOTICE.md` にあります。